



MNS UNIVERSITY OF AGRICULTURE MULTAN
DATABASE MANAGEMENT SYSTEMS
Project Report — Group 6

Online Food Ordering and Delivery Database

A fully normalised MySQL relational database system

Field	Details
Group Number	Group 6
Course	Database Management Systems
Professor Name	Mam Sania Ayoub
Submission Date	08-June-2026

#	Name	Role
1	Hussain Shah	Admin / Group Lead
2	Aqsa Aslam	Member
3	Malik Abdul Rahman	Member
4	Muhammad Talal	Member
5	Muhammad Huzaifa	Member

Table of Contents

Table of Contents	3
1. Project Overview	4
1.1 Objectives	4
1.2 Technology Stack	4
2. Database Design	5
2.1 Entity Summary	5
2.2 Relationships	5
3. Table Descriptions	6
3.1 customers	6
3.2 restaurants	6
3.3 orders — Central Table	6
Order Status Pipeline	7
3.4 order_items	8
3.5 feedback	8
4. Views	9
4.1 vw_customer_order_history	9
4.2 vw_order_details	9
4.3 vw_restaurant_ratings	9
4.4 vw_delivery_performance	9
5. Stored Procedures	10
5.1 sp_place_order	10
5.2 sp_add_order_item	10
5.3 sp_update_order_status	10
5.4 sp_assign_delivery	10
5.5 sp_submit_feedback	11
5.6 sp_cancel_order	11
6. Indexes	12
7. Sample Data	13
8. Setup Instructions	14
8.1 Requirements	14
8.2 Running the Script	14
8.3 Verification	14
9. Business Rules & Constraints	15
10. Submitted Files	16
11. Conclusion	16

1. Project Overview

This project presents the design and implementation of a fully normalised relational database for an Online Food Ordering and Delivery platform. The database is built in MySQL 8.0 and covers the complete lifecycle of a customer order — from browsing a restaurant menu and placing an order, through preparation and delivery, to final feedback submission.

The system supports multiple restaurants operating across different cities, each with their own categorised menus, independent delivery personnel, and customer accounts. Every major operation is encapsulated within stored procedures to enforce business rules at the database level.

Scope: Customers · Restaurants · Menu Management · Order Processing · Delivery Tracking · Feedback & Ratings

1.1 Objectives

- Design a fully normalised (3NF) relational database for a food ordering platform
- Model all core entities: customers, restaurants, menu items, orders, delivery personnel, and feedback
- Implement stored procedures to handle all major business operations
- Create views to simplify common reporting and query tasks
- Apply indexes to optimise query performance on high-traffic columns
- Populate the database with realistic sample data for demonstration and testing

1.2 Technology Stack

Component	Details
Database Engine	MySQL 8.0 (utf8mb4, unicode_ci collation)
Query Language	SQL — DDL, DML, Stored Procedures, Views
Development Tool	MySQL Workbench 8.0
Platform	Windows / Linux
Script File	online_food_ordering.sql (single self-contained file)

2. Database Design

The schema follows Third Normal Form (3NF) to eliminate data redundancy and ensure referential integrity. Eight tables are organised around the central Orders table, which links customers, restaurants, menu items, and delivery personnel together.

2.1 Entity Summary

#	Table	Sample Rows	Purpose
1	customers	6	Registered customer accounts
2	restaurants	5	Restaurant profiles and operating hours
3	menu_categories	10	Item groupings per restaurant
4	menu_items	18	Individual food/drink items with pricing
5	delivery_personnel	5	Riders and drivers for deliveries
6	orders	6	Master record for every placed order
7	order_items	11	Line items (menu items per order)
8	feedback	3	Customer ratings and written reviews

2.2 Relationships

Parent Table	Cardinality	Child Table	Description
customers	1 — many	orders	A customer places many orders
restaurants	1 — many	orders	A restaurant receives many orders
delivery_personnel	1 — many	orders	A rider delivers many orders (optional)
orders	1 — many	order_items	An order contains one or more items
menu_items	1 — many	order_items	An item can appear in many orders
restaurants	1 — many	menu_categories	A restaurant has many categories
menu_categories	1 — many	menu_items	A category groups many items
orders	1 — one	feedback	Each order gets at most one review
customers	1 — many	feedback	A customer submits many reviews
delivery_personnel	1 — many	feedback	A rider is rated in many reviews

3. Table Descriptions

3.1 customers

Stores registered customer accounts. Passwords are stored as SHA-256 hashes. The `is_active` flag enables soft deletion without losing order history.

Column	Type	Key	Description
customer_id	INT	PK	Auto-increment primary key
full_name	VARCHAR(100)		Customer full name
email	VARCHAR(150)	UQ	Unique email address
phone	VARCHAR(20)		Contact number
address	VARCHAR(255)		Delivery address
city	VARCHAR(80)		City
password_hash	VARCHAR(255)		SHA-256 hashed password
is_active	TINYINT(1)		Account status (1 = active)
created_at	DATETIME		Registration timestamp

3.2 restaurants

Stores restaurant profiles with operating hours and a dynamically maintained average food rating that updates automatically whenever new feedback is submitted.

Column	Type	Key	Description
restaurant_id	INT	PK	Auto-increment primary key
name	VARCHAR(150)		Restaurant name
cuisine_type	VARCHAR(80)		Cuisine type (e.g. Pakistani, Chinese)
email	VARCHAR(150)	UQ	Unique email address
opening_time	TIME		Daily opening time
closing_time	TIME		Daily closing time
is_active	TINYINT(1)		Active / inactive flag
rating	DECIMAL(3,2)		Average food rating (auto-updated)

3.3 orders — Central Table

The core table of the schema. Every order tracks its full lifecycle: placement, status pipeline, financial totals (subtotal + 5% tax + delivery fee), payment method, and delivery assignment.

Column	Type	Key	Description
order_id	INT	PK	Auto-increment primary key
customer_id	INT	FK	References customers
restaurant_id	INT	FK	References restaurants

personnel_id	INT (nullable)	FK	References delivery_personnel
subtotal	DECIMAL(10,2)		Sum of all line item totals
delivery_fee	DECIMAL(6,2)		Fixed PKR 50
tax_amount	DECIMAL(6,2)		5% of subtotal
total_amount	DECIMAL(10,2)		subtotal + fee + tax
status	ENUM		pending / confirmed / preparing / ready / out_for_delivery / delivered / cancelled
payment_method	ENUM		cash / card / online_wallet
payment_status	ENUM		unpaid / paid / refunded
delivered_at	DATETIME		Set automatically when status = delivered

Order Status Pipeline

Flow: pending → confirmed → preparing → ready_for_pickup → out_for_delivery → delivered

Cancellation: Allowed only from pending or confirmed. Once preparation begins the order cannot be cancelled.

3.4 order_items

One row per menu item per order. The line_total column is a MySQL GENERATED ALWAYS AS STORED column (quantity × unit_price) — it is computed by the database and does not need to be inserted.

Column	Type	Key	Description
order_item_id	INT	PK	Auto-increment primary key
order_id	INT	FK	References orders (CASCADE DELETE)
item_id	INT	FK	References menu_items
quantity	INT		Must be > 0 (CHECK constraint)
unit_price	DECIMAL(8,2)		Price locked at the time of ordering
line_total	DECIMAL(10,2)		GENERATED: quantity × unit_price (STORED)
special_req	VARCHAR(300)		Customer item-level special request

3.5 feedback

Customer reviews submitted after delivery. A UNIQUE constraint on order_id enforces one review per order. Submitting feedback automatically recalculates restaurant and rider average ratings.

Column	Type	Key	Description
feedback_id	INT	PK	Auto-increment primary key
order_id	INT	FK / UQ	One review per order (UNIQUE)
customer_id	INT	FK	References customers
restaurant_id	INT	FK	References restaurants
personnel_id	INT	FK	References delivery_personnel (nullable)
food_rating	TINYINT		1–5 stars for food quality
delivery_rating	TINYINT		1–5 stars for delivery (optional)
comment	TEXT		Written review text
created_at	DATETIME		Submission timestamp

4. Views

Four views are defined to simplify common reporting queries. Views are virtual — they run their underlying SELECT at query time and always reflect current data.

View Name	Purpose
vw_customer_order_history	Full order history per customer including restaurant and rider details
vw_order_details	Fully itemised order breakdown — one row per line item
vw_restaurant_ratings	Aggregated orders, reviews, and average ratings per restaurant
vw_delivery_performance	Total deliveries, completed count, and average rating per rider

4.1 vw_customer_order_history

Joins orders with customers, restaurants, and delivery_personnel. Returns the complete history for every customer — ideal for a My Orders portal page.

```
SELECT * FROM vw_customer_order_history WHERE customer_id = 1;
```

4.2 vw_order_details

One row per line item per order, fully joined with customer, restaurant, and menu item details. Useful for generating order receipts or admin dashboards.

```
SELECT * FROM vw_order_details WHERE order_id = 3;
```

4.3 vw_restaurant_ratings

Aggregates total orders placed, total reviews received, average food rating, and average delivery rating for every restaurant.

```
SELECT * FROM vw_restaurant_ratings ORDER BY avg_food_rating DESC;
```

4.4 vw_delivery_performance

Shows each rider's total assigned deliveries, successfully completed deliveries, and average delivery rating. Useful for rider performance monitoring.

```
SELECT * FROM vw_delivery_performance ORDER BY total_deliveries DESC;
```

5. Stored Procedures

Eight stored procedures encapsulate all business operations. They validate input, enforce rules, and maintain data integrity so the application layer cannot bypass database constraints.

Procedure	Description
sp_place_order	Creates a new order; returns new order_id via OUT parameter
sp_add_order_item	Adds a menu item to an open order and recalculates all totals
sp_recalculate_order_total	Internal helper — recomputes subtotal, 5% tax, and grand total
sp_update_order_status	Advances order through pipeline; blocks invalid transitions
sp_assign_delivery	Assigns an available rider; marks them unavailable automatically
sp_submit_feedback	Saves review; auto-updates restaurant and rider ratings
sp_get_menu	Returns all available items for a restaurant, grouped by category
sp_cancel_order	Cancels a pending/confirmed order; releases the assigned rider

5.1 sp_place_order

Creates a new order record. Sets a fixed delivery fee of PKR 50. The new order_id is returned via an OUT parameter so items can be added immediately.

```
CALL sp_place_order(1, 2, 'House 12, Gulshan', 'Karachi', 'cash', 'No onions',
@oid);
SELECT @oid; -- use returned id to add items
```

5.2 sp_add_order_item

Adds a menu item to an existing order. Automatically fetches the current price and recalculates subtotal, tax (5%), and total. Raises an error if the order is past the confirmed stage or the item is unavailable.

```
CALL sp_add_order_item(@oid, 5, 2, 'Extra spicy');
-- Parameters: (order_id, item_id, quantity, special_request)
```

5.3 sp_update_order_status

Enforces valid pipeline transitions. Invalid jumps raise SQLSTATE 45000. Automatically stamps delivered_at = NOW() when status reaches delivered.

- pending → confirmed or cancelled
- confirmed → preparing or cancelled
- preparing → ready_for_pickup
- ready_for_pickup → out_for_delivery
- out_for_delivery → delivered

```
CALL sp_update_order_status(10, 'confirmed');
```

5.4 sp_assign_delivery

Assigns an available delivery person to an order. Checks `is_available = 1` before assigning, then sets it to 0 for the duration of the delivery.

```
CALL sp_assign_delivery(10, 3); -- (order_id, personnel_id)
```

5.5 `sp_submit_feedback`

Saves a customer review. Validates the order belongs to the customer and is delivered. After saving, automatically recalculates the restaurant's and rider's average ratings via `UPDATE`.

```
CALL sp_submit_feedback(10, 1, 5, 4, 'Great food, fast delivery!');  
-- Parameters: (order_id, customer_id, food_rating, delivery_rating, comment)
```

5.6 `sp_cancel_order`

Cancels an order if it is pending or confirmed; any other status raises an error. Automatically resets the assigned rider's `is_available` back to 1.

```
CALL sp_cancel_order(10, 1); -- (order_id, customer_id)
```

6. Indexes

Nine indexes are applied to the most frequently queried columns to improve performance on filtering and sorting operations that would otherwise require full table scans.

Index	Table	Column	Optimises
idx_orders_customer	orders	customer_id	Customer order lookup
idx_orders_restaurant	orders	restaurant_id	Restaurant order lookup
idx_orders_personnel	orders	personnel_id	Rider delivery lookup
idx_orders_status	orders	status	Filter by status
idx_orders_date	orders	order_date	Sort / filter by date
idx_menu_items_rest	menu_items	restaurant_id	Fetch menu by restaurant
idx_menu_items_cat	menu_items	category_id	Fetch menu by category
idx_feedback_rest	feedback	restaurant_id	Rating aggregation
idx_order_items_order	order_items	order_id	Fetch line items per order

7. Sample Data

Realistic sample data spanning multiple Pakistani cities is bundled directly in the SQL script. All records are ready for immediate testing and demonstration after running the script.

Table	Records	Notes
customers	6	Karachi, Lahore, Islamabad, Rawalpindi
restaurants	5	Burning Brownie · Spice Garden · Pizza Point · Wok Express · Desi Delight
menu_categories	10	2 categories per restaurant on average
menu_items	18	Burgers, biryani, karahi, pizza, pasta, BBQ, tikka, nihari, soup
delivery_personnel	5	Mix of bike, scooter, and bicycle riders
orders	6	Statuses: delivered (3), out_for_delivery, preparing, pending
order_items	11	Covers all 6 sample orders
feedback	3	Reviews for the 3 delivered orders; ratings 4–5 stars

8. Setup Instructions

8.1 Requirements

- MySQL 8.0 or later
- MySQL Workbench 8.0 (recommended) or any compatible MySQL client
- Minimum 50 MB free disk space

8.2 Running the Script

Method 1 — MySQL Workbench:

1. Open MySQL Workbench and connect to your local MySQL server
2. Go to File → Open SQL Script and select `online_food_ordering.sql`
3. Click the lightning bolt `Execute` button to run the full script
4. The results panel will print `'=== Database setup complete ==='` at the end

Method 2 — Command Line:

```
mysql -u root -p < online_food_ordering.sql
```

8.3 Verification

The script automatically runs a row-count check at the end. Expected output:

Table	Expected Rows
customers	6
restaurants	5
menu_categories	10
menu_items	18
delivery_personnel	5
orders	6
order_items	11
feedback	3

9. Business Rules & Constraints

All rules below are enforced at the database level through constraints and stored procedure logic. They cannot be bypassed by the application layer.

#	Rule	Enforcement
1	One feedback per order	UNIQUE constraint on feedback.order_id
2	Feedback only on delivered orders	Validated in sp_submit_feedback
3	Items only added to open orders	Status check in sp_add_order_item
4	Cancel only when pending or confirmed	Status check in sp_cancel_order
5	Tax fixed at 5% of subtotal	Computed in sp_recalculate_order_total
6	Delivery fee fixed at PKR 50	Hardcoded in sp_place_order
7	Rider marked unavailable when assigned	UPDATE in sp_assign_delivery
8	Rider released when order cancelled	UPDATE in sp_cancel_order
9	Ratings auto-updated after feedback	UPDATE in sp_submit_feedback
10	Invalid status transitions blocked	IF-chain in sp_update_order_status
11	Passwords stored as SHA-256 hashes	SHA2() applied at INSERT time
12	Item price locked at time of order	unit_price copied from menu_items on insert

10. Submitted Files

File	Description
online_food_ordering.sql	Complete MySQL script — DDL, DML, stored procedures, views, indexes, sample data, and verification queries
README.md	Full technical documentation — setup instructions, table descriptions, procedure usage
Project_Report.docx	This document — formal project report for teacher submission
ER_Diagram	Entity-Relationship diagram — all 8 entities and 11 relationships with crow-foot notation

11. Conclusion

This project successfully delivers a fully normalised, production-ready relational database for an online food ordering and delivery system using MySQL 8.0. The schema covers all required entities — customers, restaurants, menus, orders, delivery personnel, and feedback — connected through well-defined foreign key relationships with appropriate ON DELETE and ON UPDATE rules.

The use of stored procedures ensures all business logic is enforced consistently at the database level. Views provide reusable query interfaces for common reporting, and nine indexes on high-traffic columns ensure the system remains performant as data grows.

The project fully meets the requirements of the course brief: it models data for customers, orders, menu items, and delivery personnel; provides views for customer order history; and implements stored procedures for order updates. The schema is immediately ready for integration with any web or mobile application front-end.

Group 6 — Database Management Systems

Hussain Shah · Aqsa Aslam · Malik Abdul Rahman · Muhammad Talal · Muhammad
Huzafa
June 2026